PHÂN TÍCH THUẬT TOÁN + RELLIS-3D

TASK 1.1:

1. Grid-map

- **Thuật toán áp dụng:** A*

- **Input**:
  + **Bản đồ 2.5D:** Một ma trận lưới, trong đó mỗi ô chứa một giá trị độ cao.
  + **Tọa độ Điểm đầu và Điểm cuối:** Vị trí Start (x_st, y_st) và Goal (x_g, y_g).
  + **Các giới hạn di chuyển của Robot:**
    + **Độ cao bước tối đa:** Độ chênh lệch độ cao lớn nhất mà robot có thể leo lên hoặc bước xuống mà không bị lật.
    + **Độ dốc tối đa:** Góc nghiêng lớn nhất cho phép đi qua.
    + **Vùng không an toàn:** Các giá trị độ cao đại diện cho vùng chưa khám phá (unknown) hoặc vật cản tuyệt đối (ví dụ: tường cao).
  + **Hàm chi phí:**
    + **g(n):** Chi phí thực tế đi từ Start đến ô hiện tại n.
    + **h(n):** Chi phí ước lượng (Heuristic) từ ô n đến Goal.

- **Output:**
  + **Đường đi tối ưu:** Một tập hợp các tọa độ liền kề tạo thành tuyến đường an toàn, trơn tru từ Start đến Goal: [(x0, y0), (x1, y1), ..., (xn, yn)]
  + **Hoặc Cờ thất bại:** Trả về khi điểm Goal bị bao quanh bởi địa hình robot không thể vượt qua (quá dốc, quá cao) và không có đường nào hợp lệ.

- **Cách chạy của thuật toán:** Thuật toán sử dụng một hàm đánh giá tổng quát f(n) = g(n) + h(n) để ưu tiên duyệt các ô có khả năng dẫn đến đích nhanh nhất và tốn ít sức nhất.
  + **Bước 1: Khởi tạo**
    + Tạo **Open List** (Danh sách các ô đang chờ duyệt - thường dùng Priority Queue) và **Closed List** (Danh sách các ô đã duyệt xong).

+ Đưa điểm Start vào Open List. Khởi tạo g(Start) = 0 và tính h(Start).

+ **Bước 2: Vòng lặp tìm kiếm**

  Lặp lại các hành động sau cho đến khi Open List rỗng:

  + **Chọn Node:** Rút ô hiện tại có giá trị f(n) thấp nhất từ Open List ra (gọi là ô Current).
  + **Kiểm tra đích:** Nếu ô `Current` chính là Goal, thuật toán dừng lại. Truy vết ngược lại các ô cha (parent) để xuất ra đường đi hoàn chỉnh.
  + **Đưa vào danh sách đã đóng:** Đưa ô `Current` vào Closed List để không duyệt lại.
  + **Mở rộng lân cận (Expand Neighbors):** Duyệt qua các ô liền kề xung quanh ô `Current` (thường là 8 hướng). Với mỗi ô kề (gọi là ô `Next`).

- *Bỏ qua nếu:* Ô `Next` đã nằm trong Closed List.
- **Kiểm tra đặc thù 2.5D (Tính hợp lệ của địa hình):** Tính độ chênh lệch độ cao: $\Delta z = |z_{next} - z_{current}|$.
  - Nếu $\Delta z > \text{Max Step Height}$, ô `Next` được coi là một bức tường/vực thẳm không thể qua. Bỏ qua ô này.
- **Tính toán chi phí $g(n)$ mới:** Đây là điểm khác biệt lớn của 2.5D. Chi phí không chỉ là khoảng cách 2D, mà phải cộng thêm hình phạt (penalty) khi leo dốc. Một công thức phổ biến là:

$$g(Next) = g(Current) + \text{Distance}(Current, Next) + w \cdot \Delta z$$

(Trong đó w là trọng số cấu hình. Trọng số càng cao, thuật toán càng ưu tiên đường bằng phẳng thay vì đi đường chim bay nhưng phải leo dốc).

- **Cập nhật Open List:** Nếu ô `Next` chưa có trong Open List, hoặc chi phí $g(Next)$ vừa tính toán thấp hơn chi phí cũ của chính nó trong Open List, ta cập nhật lại chi phí mới cho `Next`, gán `Current` làm cha của nó, và đẩy (hoặc cập nhật) nó vào Open List.

+ **Bước 3: Kết thúc**

  Nếu Open List rỗng mà vẫn chưa đến được Goal, thuật toán kết thúc và báo cáo không tìm thấy đường.

2. MLS

- **Thuật toán áp dụng:** A*

- **Input:**
  + Bản đồ MLS: Một lưới 2D, trong đó mỗi ô chứa một danh sách các "Patch" (bề mặt). Mỗi Patch lưu trữ:
    + Giá trị độ cao z (và có thể kèm theo phương sai nếu xét đến nhiễu cảm biến).
    + Độ dày của bề mặt (Depth)
  + Trạng thái Start và Goal: Tọa độ lúc này không chỉ là (x, y) mà phải chỉ định rõ bề mặt nào: (x, y, p_start) và (x, y, p_goal), với p là chỉ số của patch.
  + Các giới hạn di chuyển của Robot:
    + Độ cao bước tối đa (Max Step Height) và Độ dốc tối đa (Max Slope).
    + **Chiều cao của robot (Robot Height):** Đây là thông số *bắt buộc phải có* trên MLS để đảm bảo robot không bị đụng đầu vào cấu trúc bên trên.
  + Hàm chi phí:
    + g(n): Chi phí thực tế
    + h(n): Chiheuristic (thường dùng khoảng cách Euclidean 3D).

- **Output:**
  + Đường đi an toàn: Một tập hợp các trạng thái liên tiếp đi qua các bề mặt: [(x0, y0, p0), (x1, y1, p1), ..., (xn, yn, pn)].
  + Hoặc báo lỗi không tìm thấy đường nếu không có tuyến nào thỏa mãn các điều kiện về khoảng không và giới hạn leo dốc.

- **Cách chạy thuật toán:** Về khung thuật toán, A* vẫn sử dụng danh sách Open List, Closed List và ưu tiên duyệt các node có f(n) = g(n) + h(n) thấp nhất. Sự khác biệt mang tính quyết định nằm ở **Bước mở rộng lân cận (Expand Neighbors)**.
  + Bước 1 & 2: Khởi tạo và Rút Node (Một "Node" bây giờ đại diện cho một Patch cụ thể).
  + Bước 3: Mở rộng lần cận
    Khi đang đứng ở ô (x, y) trên bề mặt p_current, thuật toán xét các ô liền kề (x_next, y_next). Tuy nhiên, ở ô kề này có thể có rất nhiều bề mặt khác nhau (ví dụ: mặt đường, mặt cầu vượt).
    Với mỗi bề mặt p_next nằm trong ô kề (x_next, y_next), A* phải thực hiện chuỗi kiểm tra gắt gao sau:
    + **Kiểm tra khả năng bước lên:** Tính độ chênh lệch Delta z = |z(p_{next}) - z(p_{current})|. Nếu Delta z > Max Step Height, bề mặt này không thể với tới -> Bỏ qua.
    + **Kiểm tra khoảng không phía trên (Clearance - Độc quyền của MLS):** Nếu robot bước sang p_next, liệu nó có bị kẹt đầu bởi một bề mặt nào đó nằm ngay phía trên p_next hay không? / Thuật toán quét danh sách các patch tại (x_next, y_next), tìm patch nằm ngay trên p_next (gọi là p_ceiling). Khoảng cách từ mặt của p_next lên tới đáy của p_ceiling phải **lớn hơn chiều cao của robot**. Nếu nhỏ hơn, robot sẽ kẹt -> Bỏ qua bề mặt này.
    + **Tính toán chi phí g(n):** Nếu bề mặt p_next hợp lệ (cả về khả năng leo và không bị đụng đầu), chi phí di chuyển sẽ được tính toán bằng cách kết hợp khoảng cách ngang và chi phí phạt khi thay đổi độ cao, tương tự như 2.5D.
    + **Cập nhật Open List:** Nếu rẻ hơn, cập nhật node cha và đưa p_next vào Open List.

3. Mesh

- **Mesh** (hay Render/Collision Mesh) là mô hình 3D thực tế của môi trường, còn **NavMesh** (Navigation Mesh) là một bản đồ đã

được "chưng cất" lại, chỉ giữ lại những bề mặt mà robot **thực sự có thể đi lên được**.

- Quá trình chuyển đổi từ Mesh sang NavMesh thường được gọi là **Baking (Nướng bản đồ)**. Dưới đây là lý do tại sao phải chuyển đổi và cách quá trình này diễn ra.
- **Thuật toán áp dụng:** A*

- Tại sao không chạy thẳng A* trên Mesh gốc?
  + **Quá nhiều chi tiết thừa:** Một Mesh 3D của căn phòng có thể chứa hàng triệu tam giác để vẽ chi tiết từng nếp nhăn của nệm ghế, từng viên sỏi trên sàn. Thuật toán A* sẽ bị quá tải nếu phải duyệt qua ngần ấy node.
  + **Không phân biệt được chỗ đi được và không đi được:** Trọng tâm của A* là tìm đường. Mesh gốc bao gồm cả trần nhà, bức tường thẳng đứng, mặt bàn... Nếu không xử lý, A* có thể chỉ đường cho robot... leo ngược lên tường.

- NavMesh là gì?

NavMesh là một lớp đồ thị phủ lên trên bề mặt của Mesh gốc. Nó là tập hợp các đa giác lồi (thường lớn và ít cạnh hơn rất nhiều so với Mesh gốc) ghép lại với nhau.

Bất cứ điểm nào nằm bên trong NavMesh đều được đảm bảo là:

+ Độ dốc nằm trong mức cho phép.
+ Không bị vướng trần nhà (đủ không gian cho chiều cao robot).
+ Không bị kẹt vào tường (đủ không gian cho chiều rộng robot).

- Quá trình "Baking" (Chuyển Mesh -> NavMesh) diễn ra như thế nào?

Hệ thống sẽ dùng một "khuôn" đại diện cho robot (gồm chiều cao, bán kính, khả năng leo dốc, khả năng bước qua chướng ngại vật) để quét qua toàn bộ Mesh 3D gốc. Quá trình này gồm các bước chính:

+ **Bước 1: Voxel hóa (Voxelization):** Hệ thống tạm thời chia toàn bộ không gian 3D thành các khối lập phương nhỏ (voxel) để dễ dàng phân tích không gian đặc/rỗng.

+ **Bước 2: Lọc tính di chuyển (Walkability Filtering):** Hệ thống loại bỏ các voxel có độ dốc cao hơn `Max Slope`, các bậc thang cao hơn `Max Step Height`, và những nơi khoảng không phía trên thấp hơn `Robot Height` (giống y hệt cách kiểm tra của MLS mà chúng ta đã bàn).

+ **Bước 3: Thu hẹp ranh giới (Erosion / Shrinking):** Đây là bước cực kỳ quan trọng. Ranh giới của các vùng đi được sẽ bị "bóp" thu hẹp lại một khoảng đúng bằng **Bán kính của Robot**.
  Note: *Tại sao phải làm vậy?* Nhờ thu hẹp bản đồ lại, khi A* tính toán, nó chỉ cần coi robot là **một điểm duy nhất**. Điểm đó có thể đi sát ra tận rìa của NavMesh mà phần thân thực tế của robot vẫn không bị cạ vào bức tường thật.

+ **Bước 4: Tái tạo đa giác (Polygonization):** Từ các bề mặt hợp lệ còn sót lại, hệ thống sẽ ghép chúng thành các đa giác lồi lớn và đơn giản nhất có thể. Thành phẩm cuối cùng chính là NavMesh.

- **Input:**
  + **Bản đồ đa giác (Mesh Graph):** Tập hợp các đỉnh (Vertices) tọa độ 3D và các mặt đa giác (Faces/Polygons). Thông tin quan trọng nhất ở đây là **tính kề nhau (Adjacency)** – hệ thống phải biết rõ tam giác nào đang chia sẻ cạnh với tam giác nào.
  + **Tọa độ Start và Goal:** Là tọa độ không gian liên tục (x, y, z) thay vì chỉ mục hàng/cột như trên Grid.
  + **Thông số Robot:** Bán kính và chiều cao. (Thông thường, bản đồ Mesh thực tế đã được làm hẹp lại một khoảng bằng bán kính robot để đảm bảo robot chỉ cần coi như một điểm vật lý di chuyển trên lưới mà không lo va quệt thành tường).
  + **Giới hạn cấu hình:** Độ dốc tối đa (Max Slope) giữa 2 tam giác kề nhau.
  + **Hàm chi phí:**
    + g(n): Khoảng cách di chuyển qua các đa giác.
    + h(n): Khoảng cách đường chim bay Euclidean.

- **Output:**
  Khác với Grid/MLS xuất ra các tâm ô vuông, đầu ra của A* trên Mesh bao gồm 2 phần:
  + **Đường ống đa giác:** Một chuỗi các đa giác liền kề nhau dẫn từ điểm bắt đầu đến điểm kết thúc.
  + **Đường đi chính xác:** Một tập hợp các tọa độ (x, y, z) trơn tru và tối ưu nhất nằm *bên trong* đường ống đa giác đó.

- **Cách chạy của thuật toán:**
  Trên bản đồ Mesh, một "Node" (Nút) được xét trong thuật toán A* không phải là một tọa độ, mà là **một đa giác (tam giác)** hoặc **điểm giữa của cạnh chung** giữa hai tam giác.
  + **Bước 1: Định vị ban đầu**
    Thuật toán không thể chạy ngay. Trước tiên, nó phải dùng các phép toán hình học để tìm xem tọa độ liên tục Start và Goal đang nằm chính xác trên tam giác nào trong hàng ngàn tam giác của đồ thị. Ký hiệu là T_start và T_goal.

  + **Bước 2: Tìm "đường ống" bằng A***
    - Đưa T_start vào Open List.
    - Vòng lặp rút Node có f(n) = g(n) + h(n) nhỏ nhất (gọi là T_current). Ở đây, khoảng cách g(n) thường được đo từ tâm của tam giác này đến tâm của tam giác kề nó.
    - Nếu độ chênh lệch góc dốc giữa hai tam giác vượt quá khả năng leo của robot, cạnh nối đó bị coi là không hợp lệ -> bỏ qua.
    - Nếu hợp lệ, tính toán lại chi phí g(n), cập nhật Open List và lưu lại vết (Parent) như bình thường.
    - Khi T_goal được lấy ra khỏi Open List, việc truy vết sẽ trả về một chuỗi các tam giác liền kề.

  + **Bước 3: Làm mượt đường đi (Path Smoothing - Bước bắt buộc của Mesh)**

    Đây là điểm sáng giá nhất của Mesh so với Grid. A* mới chỉ cho biết robot phải đi qua các tam giác nào. Nếu robot đi

theo kiểu nối từ tâm tam giác này sang tâm tam giác kia, quỹ đạo sẽ bị gãy khúc và dài dòng.

Lúc này, một thuật toán bổ trợ (phổ biến nhất là **Funnel Algorithm** - Thuật toán cái phễu) sẽ được kích hoạt. Nó tưởng tượng đường đi như một sợi dây cao su luồn qua chuỗi tam giác ("đường ống" vừa tìm được). Thuật toán sẽ "kéo căng" sợi dây này, khiến nó trượt và bám sát vào các đỉnh (vertices) của đa giác một cách tự nhiên nhất, tạo ra quỹ đạo cực kỳ trơn tru và tối ưu về mặt khoảng cách.

TASK 1.2:

Báo cáo Phân tích Bộ dữ liệu RELLIS-3D: Thúc đẩy Khả năng Điều hướng Tự hành trong Môi trường Địa hình Phức tạp

1. Tổng quan về Thách thức Điều hướng Ngoài đô thị

Trong kỷ nguyên của robot tự hành, việc chuyển dịch từ các đại lộ đô thị có cấu trúc sang môi trường địa hình (off-road) không chỉ là sự thay đổi về bối cảnh mà là một cuộc cách mạng về yêu cầu nhận thức. Các hệ thống dẫn đường hiện nay, vốn phụ thuộc nặng nề vào LiDAR hình học và Hệ thống Dẫn đường Quán tính (INS), đang đối mặt với một "nút thắt về nhận thức ngữ cảnh" (cognitive bottleneck) nghiêm trọng.

Sự hạn chế cốt lõi của các phương pháp tiếp cận thuần túy hình học nằm ở việc không thể phân biệt được khả năng di chuyển (traversability) dựa trên bản chất vật liệu. Một bụi rậm thưa có thể được LiDAR phân loại là vật cản cứng do có khối tích, nhưng thực tế một robot hạng nặng như Warthog có thể đi xuyên qua dễ dàng. Ngược lại, một vũng bùn lầy hoặc mặt nước có thể trông "phẳng" về mặt hình học nhưng lại là tác nhân gây sa lầy hoặc hư hỏng hệ thống. Các bộ dữ liệu đô thị kinh điển như KITTI hay Cityscapes hoàn toàn không thể chuyển đổi trực tiếp sang môi trường này do ranh giới đối tượng không xác định (unstructured boundaries) và sự biến thiên địa hình cực đoan. Bộ dữ liệu RELLIS-3D được thiết kế để giải quyết

khoảng trống này, cung cấp một hạ tầng dữ liệu đa phương thức (multimodal) nhằm xây dựng các hệ thống nhận thức robot có khả năng hiểu sâu sắc về thực địa.

2. Nền tảng Robot Warthog và Cấu hình Hệ thống Cảm biến

Nghiên cứu sử dụng Clearpath Robotics Warthog, một nền tảng robot lội nước mạnh mẽ, được thiết kế cho các nhiệm vụ off-road khắc nghiệt. Chiến lược thu thập dữ liệu tập trung vào sự đồng bộ hóa tuyệt đối giữa các cảm biến để tối ưu hóa khả năng fusion dữ liệu.

Bảng thông số hệ thống cảm biến RELLIS-3D

Thiết bị	Loại cảm biến	Vị trí lắp đặt & FOV	Thông số vận hành
Ouster OS1	LiDAR	Đỉnh khung cảm biến (FOV dọc 45°)	64 kênh, 10 Hz, độ phân giải 2048
Velodyne Ultra Puck	LiDAR	Dưới OS1 (FOV dọc 40°)	32 kênh, 10 Hz
Basler acA1920-50gc	Camera RGB	Mặt trước, trung tâm	1920x1200, 10 Hz, ống kính 16mm
Nerian Karmin2	Stereo Camera	Dưới camera Basler	3D Stereo, 10 Hz
Vectornav VN-300	GPS/IMU	Phía sau khung cảm biến	GPS 300 Hz, IMU 100 Hz

Đánh giá sự đồng bộ và tính tái lập

Hệ thống thiết lập một chuẩn mực mới về độ chính xác thời gian bằng cách sử dụng Giao thức Thời gian Chính xác (PTP), được coi là "tiêu chuẩn vàng" trong các hệ thống robot đa cảm biến. Việc sử dụng PTP đảm bảo sai số thời gian giữa các thiết bị Ethernet (LiDAR, Camera) ở mức nano giây. Đối với Vectornav VN-300 – thiết bị

không hỗ trợ PTP – sự đồng bộ được duy trì thông qua logic chọn gói dữ liệu có timestamp gần nhất từ luồng cập nhật tần số cao (100Hz/300Hz), đảm bảo sai số đồng bộ trong ngưỡng chấp nhận được cho các nhiệm vụ di chuyển nhanh. Toàn bộ dữ liệu được cung cấp dưới định dạng **ROS bag**, một yếu tố then chốt để đảm bảo tính tái lập (reproducibility) trong thực nghiệm khoa học.

3. Đặc điểm và Cấu trúc Bộ dữ liệu RELLIS-3D

RELLIS-3D thiết lập một hệ thống phân loại (ontology) gồm 20 lớp đối tượng, tập trung đặc biệt vào các đặc điểm quyết định khả năng vận hành của robot.

- **Phân tích Ontology:** Điểm đột phá của RELLIS-3D là việc phân tách chi tiết giữa "Puddle" (vũng nước nông - có thể đi qua) và "Water" (nước sâu - nguy hiểm). Sự phân hóa này, cùng với các lớp như "Mud" (bùn) và "Rubble" (đống đổ nát), cho phép xây dựng các mô hình đánh giá rủi ro địa hình vượt trội hơn so với các nhãn đơn giản trong đô thị.
- **Thống kê và Gán nhãn:** Dữ liệu bao gồm 13,556 quét LiDAR và 6,235 hình ảnh. Quy trình gắn nhãn LiDAR được thực hiện tinh vi bằng cách sử dụng hệ thống SLAM để đăng ký (registration) và đóng vòng lặp (loop closure), từ đó chiếu các nhãn 2D từ ảnh sang đám mây điểm 3D một cách chính xác.
- **Hệ quả của Mất cân bằng lớp (Class Imbalance):** Phân tích cho thấy sự chiếm ưu thế tuyệt đối của các lớp nền như Bầu trời, Cỏ, Cây và Bụi rậm (chiếm 94% pixel ảnh và 80% điểm LiDAR). Từ góc độ chuyên gia, đây là một thách thức chiến lược: các mô hình học sâu sẽ có xu hướng "bias" cực nặng về phía các lớp nền này. Điều này giải thích tại sao IoU của các vật cản quan trọng nhưng chiếm diện tích nhỏ như "Pole" (cột) hoặc "Fence" (rào chắn) thường tiệm cận mức 0, tạo ra những lỗ hổng an toàn chết người nếu không có các kỹ thuật bù đắp trọng số.

4. Đánh giá Hiệu năng (Benchmarks) và Kết quả Thử nghiệm

Việc đánh giá dựa trên chỉ số mIoU (mean Intersection-over-Union) cho thấy một khoảng cách hiệu năng đáng kể khi áp dụng các mô hình đô thị vào môi trường off-road.

So sánh hiệu năng các thuật toán tiêu chuẩn

Phương thức	Thuật toán	mIoU RELLIS-3D	So sánh với chuẩn Đô thị

Hình ảnh (2D)	**Gated-SCNN**	**52.92%**	Thấp hơn 21.78% so với Cityscapes
	HRNet + OCR	**51.55%**	Thấp hơn 29.55% so với Cityscapes
LiDAR (3D)	**SalsaNext**	**43.07%**	Thấp hơn 16.43% so với SemanticKITTI
	KPConv	**19.97%**	Suy giảm nghiêm trọng từ mức 58.8%

Phân tích lỗi kỹ thuật chuyên sâu

Sự sụt giảm nghiêm trọng của KPConv (xuống còn 19.97%) cho thấy các thuật toán dựa trên cấu trúc hình học cục bộ gặp khó khăn cực lớn với dữ liệu off-road không đồng nhất. Ma trận nhầm lẫn (Confusion Matrix) chỉ ra các "điểm mù" cố hữu:

1. **Sự tương đồng thị giác/hình học:** Thuật toán thường xuyên nhầm lẫn giữa *Grass* và *Bush*, hoặc *Mud* và *Puddle* do ranh giới kết cấu không rõ ràng.
2. **Lỗi nhận diện vật cản nhỏ:** Các lớp như rào chắn, cột điện và nước có tỷ lệ nhận diện sai cao do thiếu hụt dữ liệu huấn luyện (long-tail distribution), dẫn đến việc robot có thể đâm vào rào chắn vì nhầm tưởng đó là bụi rậm.

5. Phân tích Chiến lược và Định hướng Nghiên cứu

Dữ liệu thực nghiệm từ RELLIS-3D khẳng định rằng nhận thức đơn phương thức (unimodal) là không đủ để đạt được mức độ tự hành tin cậy cao.

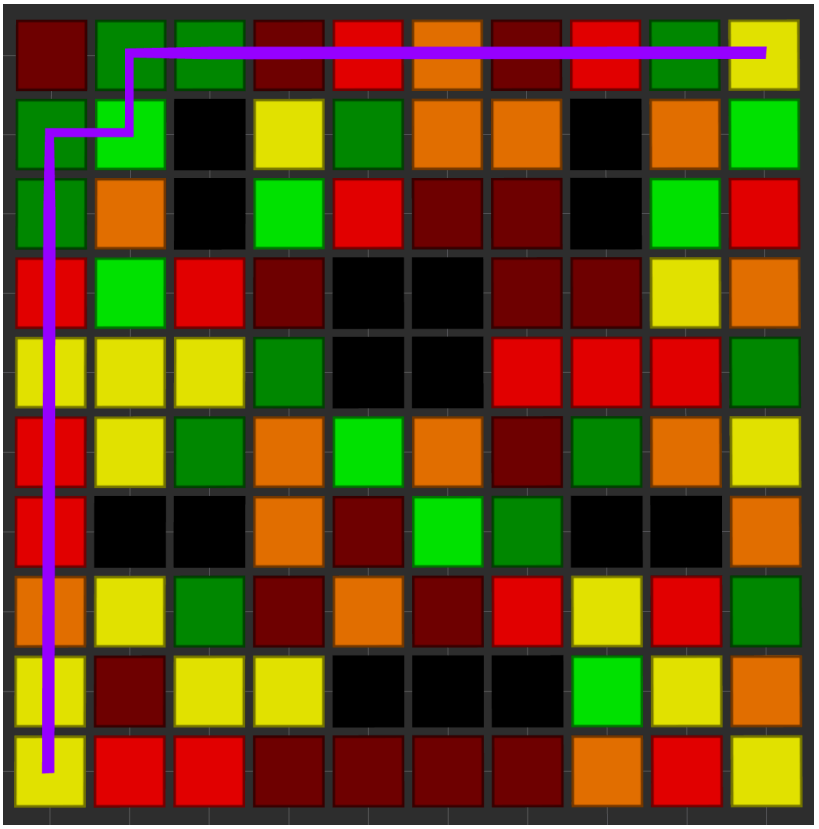
- **Giá trị của Tích hợp đa thời gian (Temporal Integration):** Phân tích hình ảnh thực tế (Fig. 7) cho thấy sức mạnh của việc kết hợp dữ liệu. LiDAR có thể "nhìn xuyên" qua các tán lá thưa để phát hiện rào chắn ẩn bên trong mà camera RGB hoàn toàn bị đánh lừa. Đặc biệt, việc sử dụng các bản quét đã đăng ký (Registered Scans) thay vì một bản quét đơn lẻ (Single Scan) cho phép tích hợp dữ liệu qua thời gian, làm nổi bật các cấu trúc vật thể bị che khuất, giúp ngăn chặn các quyết định điều hướng nguy hiểm.
- **Tầm nhìn phát triển:** Hướng đi tương lai cần tập trung vào việc chuyển đổi từ ontology dựa trên đối tượng sang ontology dựa trên khả năng di chuyển (traversability-based ontology). Việc tích hợp thêm các mốc chuẩn odometry

từ dữ liệu GPS/Stereo và gán nhãn ngữ cảnh cao cấp (như độ dốc, tính chất bề mặt) sẽ là chìa khóa cho khả năng tự hành dài hạn (Long-term Autonomy).

DEMO

1. Grid_map

```
0 0 0 0 0 0 0 0 0 0
0 0 1 0 0 0 0 1 0 0
0 0 1 0 0 0 0 1 0 0
0 0 0 0 1 1 0 0 0 0
0 0 0 0 1 1 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 1 1 0 0 0 0 1 1 0
0 0 0 0 0 0 0 0 0 0
0 0 0 1 1 1 0 0 0 0
0 0 0 0 0 0 0 0 0 0
```



1. Dữ liệu Đầu vào (Input)

Chương trình lấy dữ liệu từ hai nguồn: đọc từ file text và tự động sinh ngẫu nhiên.

- **Dữ liệu đọc từ file (input.txt):**
  - Kích thước bản đồ lưới: Chiều rộng (w) và Chiều dài (h).
  - Tọa độ: Điểm xuất phát (Start) và Điểm đích (Goal).
  - Ma trận vật cản 2D: Lưới chứa các giá trị 0 (vùng đi được) và 1 (vật cản cứng). Điểm Start và Goal được ép buộc gán bằng 0 để đảm bảo robot luôn có chỗ đứng.
- **Dữ liệu độ cao (Sinh tự động):**
  - Chương trình tự động tạo ra một mảng độ cao ngẫu nhiên (từ 0.0 đến 1.0) cho tất cả các ô trên bản đồ.
  - Đối với những ô là vật cản (giá trị 1 trong file text), hệ thống sẽ chủ động cộng thêm 1.2 vào độ cao để biến chúng thành những chướng ngại vật cực kỳ khó vượt qua.

2. Dữ liệu Đầu ra (Output)

Chương trình phát dữ liệu hình ảnh 3D lên phần mềm RViz thông qua các topic của ROS 2. Đầu ra bao gồm hai thành phần hình ảnh trực quan:

- **Bản đồ địa hình (Grid):** Hiển thị dưới dạng một mảng các ô vuông phẳng liền kề nhau.
  - Ô vật cản: Được tô màu đen thui.
  - Ô đi được: Được tô màu tự động dựa trên giá trị độ cao theo dải 6 màu: Xanh lá (thấp/bằng phẳng nhất) -> Xanh lá đậm -> Vàng -> Cam -> Đỏ -> Đỏ đậm (cao nhất/nguy hiểm nhất).
- *Đường đi A (Path):** Nếu tìm được đường, hệ thống sẽ vẽ một dải băng liên tục (Line Strip) màu tím. Dải băng này được nâng cao lên 0.1 mét so với mặt đất để không bị các ô vuông lấp mất.

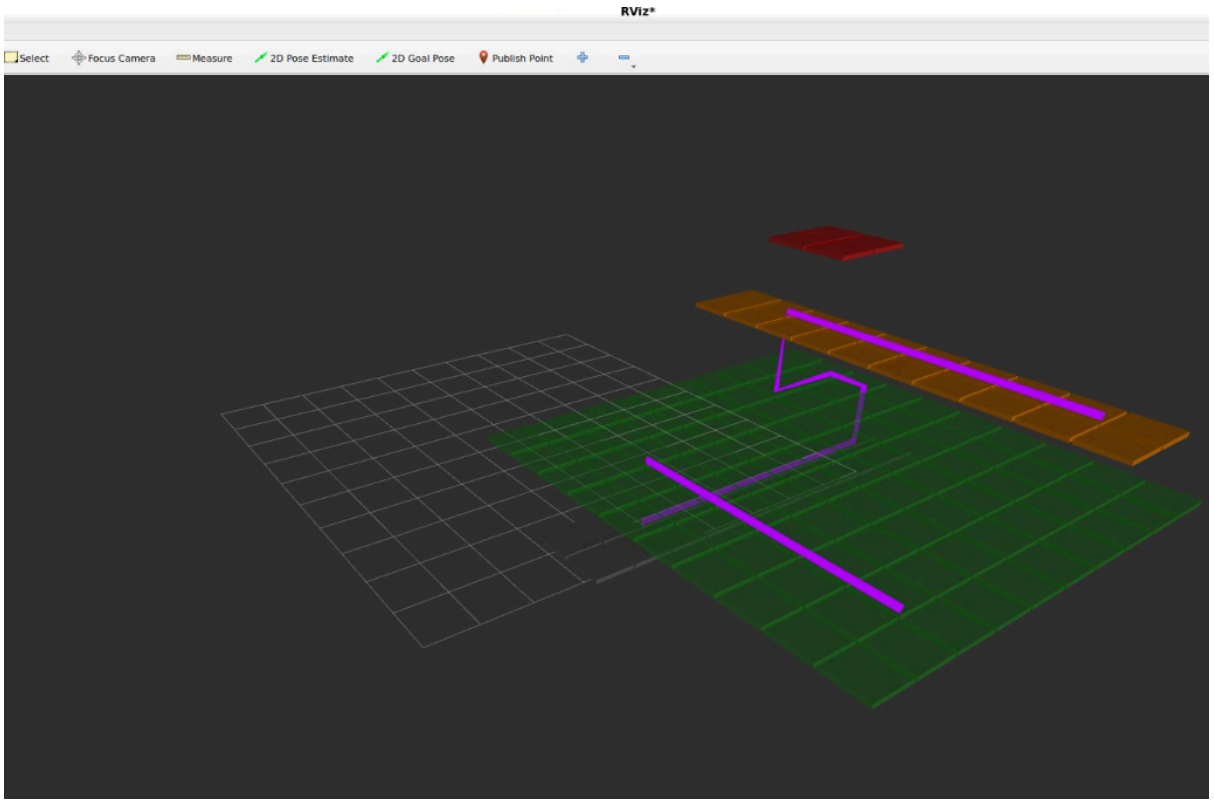
3. Quá trình di chuyển và Tìm đường (Thuật toán A*)

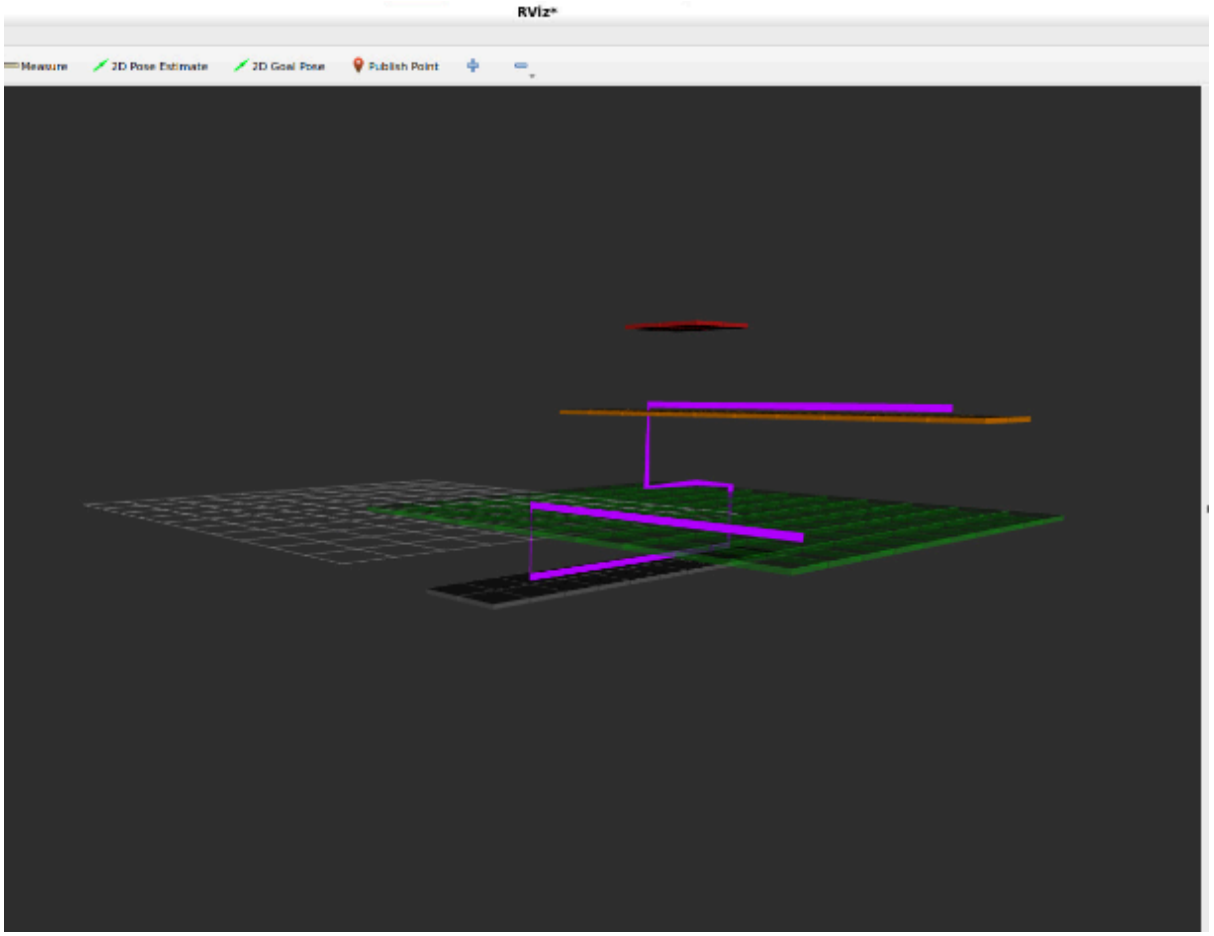
Robot di chuyển trên lưới này bằng cách duyệt qua 4 hướng cơ bản (Lên, Xuống, Trái, Phải). Quá trình tìm đường của thuật toán A* diễn ra theo các nguyên tắc cực kỳ chặt chẽ của bản đồ 2.5D:

- **Luật né vật cản cứng:** Thuật toán ngay lập tức loại bỏ và không bước vào các ô có giá trị vật cản bằng 1 (các ô màu đen).
- **Cách tính chi phí bước đi (Hàm Cost):** Đây là điểm cốt lõi của code. Khi xét bước đi từ ô hiện tại sang ô tiếp theo, chi phí không chỉ là khoảng cách 1 bước, mà bị phạt rất nặng dựa trên địa hình:
  - Chi phí cơ bản: Luôn tốn 1 đơn vị năng lượng.
  - Hình phạt độ cao (Height Cost): Lấy độ cao của ô sắp tới nhân với 5. Ô càng cao (màu cam, đỏ) thì phạt càng nặng.

○ Hình phạt độ dốc (Slope Cost): Lấy độ chênh lệch cao độ giữa ô hiện tại và ô sắp tới nhân với 10. Điều này khiến robot cực kỳ ghét việc phải leo lên hoặc nhảy xuống các bậc thềm có độ chênh lệch lớn.

2. MLS





1. Dữ liệu Đầu vào (Input)

Đầu vào của chương trình là một ma trận không gian giả lập kích thước 10 x 10, trong đó dữ liệu môi trường được thiết lập với **4 lớp bề mặt (patches)** ở các độ cao (z) khác nhau:

- **Mặt đất (Ground):** Lớp nền tảng bao phủ toàn bộ khu vực 10 x 10, nằm ở cao độ z = 0.0.
- **Hầm ngầm (Tunnel):** Một tuyến đường hầm chạy dọc theo trục x (từ x=2 đến x=3), khoét sâu xuống dưới mặt đất tại z = -1.5.
- **Cầu vượt (Bridge):** Một cây cầu chạy ngang theo trục y (từ y=6 đến y=7), bắc ngang qua mặt đất tại cao độ z = 2.0.
- **Trạm quan sát (Platform):** Nằm lơ lửng ở cao độ z = 4.0, đặt chính xác tại "ngã tư" nơi hầm ngầm và cầu vượt giao nhau. *(Đây là điểm đắt giá nhất của MLS: Tại cùng một tọa độ XY này, có tới 4 độ cao khác nhau được lưu trữ đồng thời).*

Ngoài ra, đầu vào còn chứa tập hợp tọa độ các điểm mốc (waypoints) của thuật toán A* để hướng dẫn robot đi từ Start đến Goal.

2. Dữ liệu Đầu ra (Output)

Chương trình xuất ra một luồng dữ liệu liên tục (với tần số 1 Hz) thông qua ROS 2 topic có tên là /mls_map. Dữ liệu này là một MarkerArray bao gồm hai thành phần hình học để hiển thị trên phần mềm RViz:

- **Tập hợp các khối lập phương (CUBES):** Đại diện cho các bề mặt địa hình 3D có thể đi lại được, với màu sắc và độ trong suốt khác nhau để dễ quan sát (Đất xanh lá, Hầm xám, Cầu cam, Trạm đỏ).
- **Một dải đường kẻ liền mạch (LINE_STRIP):** Đại diện cho quỹ đạo di chuyển tối ưu (màu tím), nối các tọa độ 3D lại với nhau.

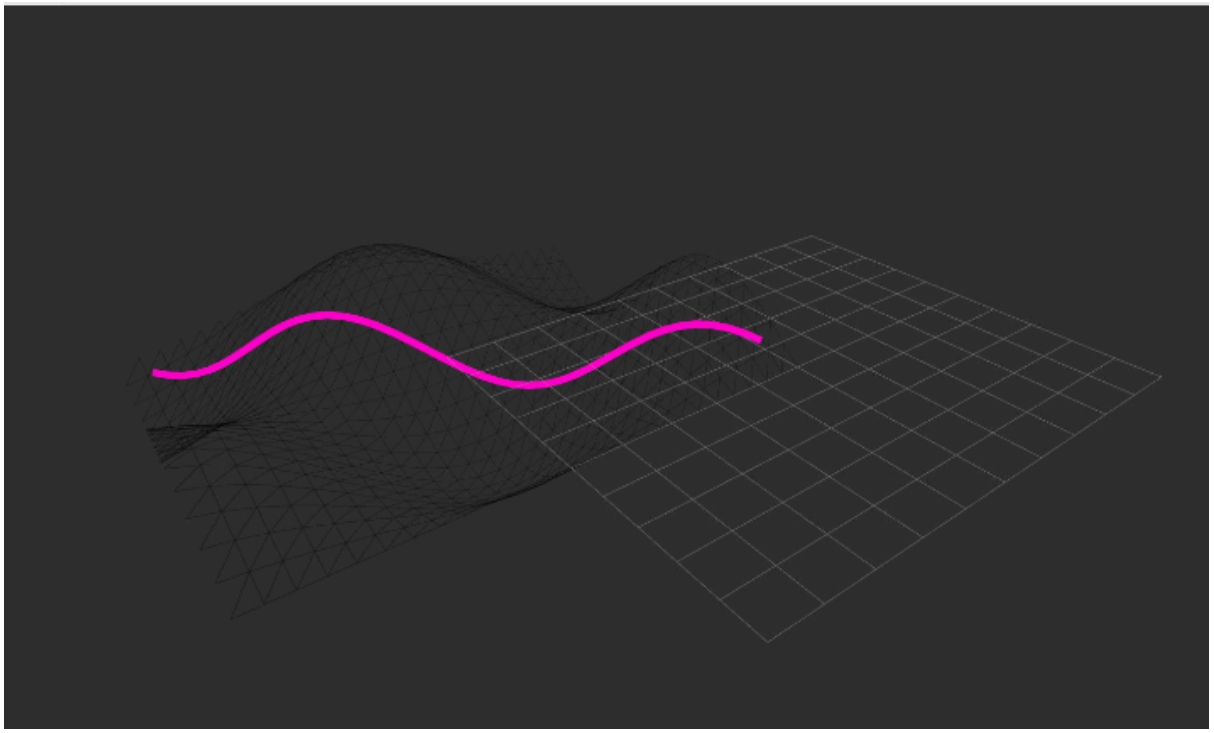
---

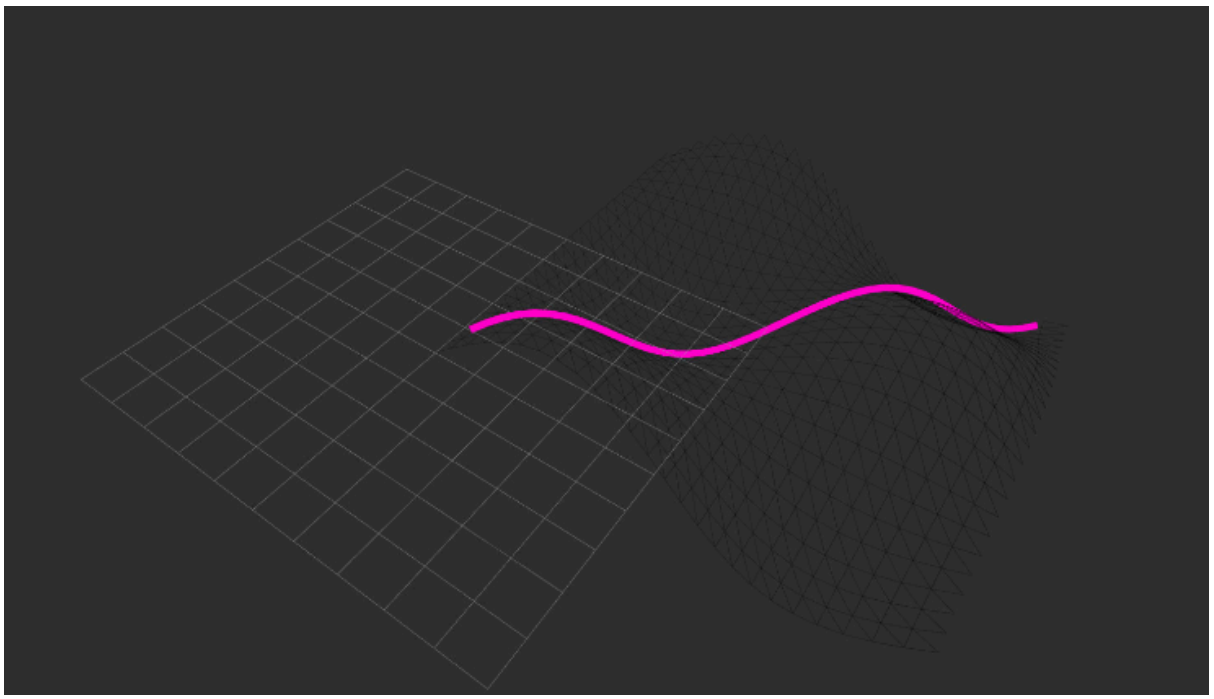
3. Quá trình di chuyển (Kịch bản tìm đường A*)

Kịch bản này mô phỏng hoàn hảo khả năng tìm đường 3D của thuật toán A* trên môi trường MLS, xử lý linh hoạt việc chui hầm và leo dốc. Quá trình diễn ra theo 5 giai đoạn:

1. **Xuất phát trên mặt đất:** Robot bắt đầu tại tọa độ (8.5, 1.5) trên mặt phẳng đất (cao độ z = 0.2 - hơi nhô lên so với mặt đất để không bị lấp).
2. **Tiến vào lòng đất:** Robot di chuyển thẳng đến miệng hầm tại (2.5, 1.5), sau đó tụt hẳn xuống không gian ngầm ở độ sâu z = -1.3.
3. **Chui qua gầm cầu và trạm:** Robot luồn lách bên dưới lòng đất theo trục dọc hầm ngầm, đi xuyên qua bên dưới cả cây cầu và trạm quan sát mà không hề gặp vật cản, đến vị trí (2.5, 8.5).
4. **Ngoi lên mặt đất và tiếp cận cầu:** Robot ngoi trở lại mặt đất tại (2.5, 8.5), thực hiện một cú rẽ trái trên mặt phẳng để tiến tới chân cầu vượt tại (1.5, 6.5).
5. **Leo lên cầu và về đích:** Cuối cùng, robot leo lên dốc cầu, thay đổi cao độ từ 0.2 lên 2.2 và đi thẳng dọc theo cây cầu để cán đích tại (8.5, 6.5).

3. Mesh





1. Dữ liệu Đầu vào (Input)

Khác với việc đọc từ file tĩnh, đầu vào của chương trình này được sinh ra tự động để tạo ra một không gian đồi núi 3D liên tục có kích thước 10 x 10 mét.

- **Dữ liệu Địa hình (Hàm độ cao):** Thay vì các ô vuông phân bậc, độ cao của từng điểm trên bản đồ được tính toán bằng một hàm lượng giác (kết hợp sin và cos). Hàm này tạo ra một bề mặt nhấp nhô, uốn lượn tự nhiên giống như những ngọn đồi và thung lũng thực tế.
- **Độ phân giải (Lưới đa giác):** Bản đồ được băm nhỏ thành các điểm cách nhau 0.4 mét. Cứ 4 điểm kề nhau sẽ được chéo hóa để tạo thành 2 tam giác, giúp ghép nối thành một bề mặt Mesh khép kín.
- **Dữ liệu Phân loại Địa hình (Màu sắc):** Đầu vào tự động phân loại bề mặt dựa trên độ cao để gán màu thực tế:
  - Dưới -0.5 mét: Vùng nước (Xanh biển)
  - Dưới 0.2 mét: Bãi cỏ (Xanh lá)
  - Dưới 0.9 mét: Vùng đất đồi (Nâu vàng)
  - Trên 0.9 mét: Đỉnh núi tuyết (Trắng)

2. Dữ liệu Đầu ra (Output)

Chương trình phát dữ liệu hình ảnh 3D liên tục (mỗi 0.5 giây) lên phần mềm RViz thông qua topic "mls_map". Để có một khung hình đẹp và không bị lỗi đè hình, đầu ra được chia làm 4 thông điệp cụ thể:

- **Lệnh Xóa:** Lệnh quét sạch mọi dữ liệu rác của khung hình trước đó để bản đồ luôn sắc nét.

- **Bề mặt đa giác đặc (Solid Mesh):** Một bề mặt đồi núi 3D trơn tru được ghép lại từ hàng trăm mặt tam giác nhỏ, có màu sắc thay đổi mượt mà theo độ cao.
- **Khung viền lưới (Wireframe):** Một mạng lưới các đường kẻ siêu mảnh, màu đen mờ, bao quanh các cạnh của tam giác. Nó giúp người xem nhìn rõ cấu trúc "lưới đa giác" cấu tạo nên bề mặt.
- **Đường đi (Path):** Một dải băng màu hồng dạ quang dày dặn, lơ lửng cách mặt đất một khoảng cực nhỏ (0.15 mét) để luôn nổi bật trên nền địa hình.

3. Quá trình di chuyển (Kịch bản NavMesh)

Quá trình này minh họa sự ưu việt của NavMesh (kết hợp với thuật toán làm mượt Funnel Algorithm) so với việc đi zíc-zắc của Grid Map 2D:

- **Quỹ đạo đi chéo tự do:** Robot xuất phát từ góc dưới cùng bên trái (tọa độ 0.5, 0.5) và nhắm thẳng đến góc trên cùng bên phải (tọa độ 9.5, 9.5). Khác với lưới ô vuông bắt buộc phải bẻ góc 90 độ, đường đi trên NavMesh đâm chéo một cách dứt khoát và ngắn nhất qua các bề mặt tam giác.
- **Bám sát địa hình 3D:** Thay vì đi thành một đường thẳng đơ giữa không trung, quỹ đạo màu hồng được chia làm 40 bước nhỏ. Tại mỗi bước, hệ thống nội suy chính xác độ cao của ngọn đồi tại tọa độ đó và uốn cong đường đi theo.
- **Kết quả di chuyển:** Robot sẽ lướt đi vô cùng mượt mà, trượt xuống thung lũng nước xanh biển, bò dốc qua các bãi cỏ xanh lá, leo lên đỉnh núi tuyết màu trắng và dừng lại ở đích đến một cách trơn tru, không có bất kỳ một góc gãy khúc nào.